BURSA TEKNİK ÜNİVERSİTESİ

MÜHENDİSLİK VE DOĞA BİLİMLERİ FAKÜLTESİ

BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

Node.JS ile Web Programlama

LAB_5

1. Not Ekleme Fonksiyonunun Geliştirilmesi

Bu aşamada addNote fonksiyonu geliştirilerek daha verimli hale getirildi. Daha önce aynı başlığa sahip notları kontrol etmek için filter metodu kullanılırken, bu hafta performans açısından daha uygun olduğu için find metodu kullanıldı. find metodu ilk eşleşmeyi bulduğu anda işlemi durdurduğu için gereksiz bellek ve işlem yükü oluşturmaz.

```
const addNote =(title,body)=>{
    const notes =loadNotes()
    const duplicateNotes=notes.find((notes)=>{
        notes.title === title
    })
    if (!duplicateNotes){
        notes.push({
            title:title,
            body:body
        })
        saveNotes(notes)
        console.log(chalk.green("not eklendi"))
        }
    else{
        console.log(chalk.bgred("not basligi mevcut"))
    }

}
```

2.Okuma Fonksiyonunun Geliştirilmesi

Bu hafta yeni olarak readNote fonksiyonu yazıldı. Bu fonksiyon, verilen başlığa göre ilgili notu bulup ekrana yazdırmaktadır.

Fonksiyon içerisinde:

- loadNotes ile veriler JSON dosyasından alınır
- find metodu ile ilgili başlık aranır
- Eğer not bulunursa başlık ve içerik ekrana yazdırılır
- Bulunamazsa kullanıcıya hata mesajı verilir

Ayrıca çıktıların daha dikkat çekici olması için chalk kütüphanesi kullanılarak renkli mesajlar gösterilmiştir.

```
const readNote = (title)=>{
    const notes=loadNotes();
    const findNote=notes.find((note) => note.title === title)

    if (findNote){
       console.log(chalk.blue(`Baslik: ${findNote.title}\n İcerik: ${findNote.body}`));}

    else{
        console.log(chalk.red("Not basliğindan bulunamadi"))
    }

}

module.exports={
    loadNotes:loadNotes,
    addNote: addNote,
    removeNotes:removeNote,
    listNotes:listNotes,
    readNote:readNote
}
```

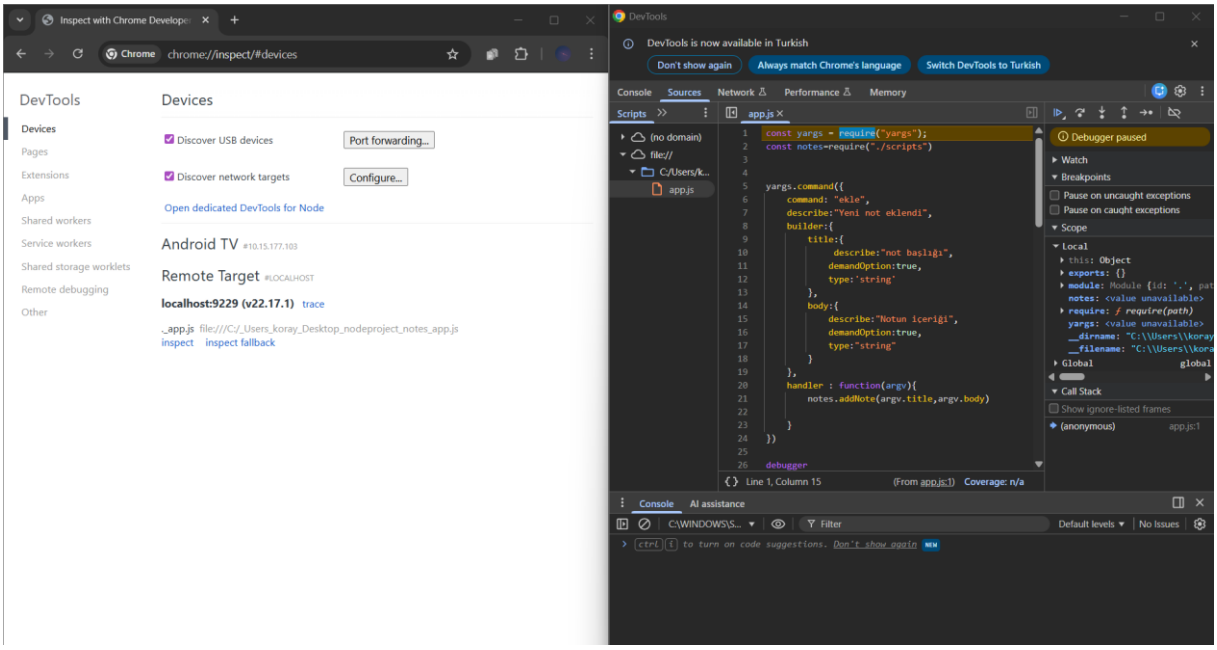
```
49  })
50  yargs.command({
51      command:"oku",
52      describe:"Basligi verilen notu okuma",
53      builder:{
54          title:{
55              describe:"not başlığı",
56              demandOption:true,
57              type:'string'
58          }},
59      handler : (argv)=>{
60         notes.readNote(argv.title)
61      }
62
63  })
64
65  yargs.parse()
66
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\koray\Desktop\nodeproject\notes> node .\app.js oku --title="rapor"
Baslik: rapor
 İcerik: lab raporu teslimi
PS C:\Users\koray\Desktop\nodeproject\notes> node .\app.js oku --title="node"
Baslik: node
 İcerik: js
PS C:\Users\koray\Desktop\nodeproject\notes>
```

3.Debugging

Bu aşamada debugging işlemleri incelendi. Kod içerisine debugger ifadesi eklenerek programın belirli bir noktada durması sağlandı. Daha sonra tarayıcı üzerinden chrome://inspect adresine girilerek Node.js uygulaması bağlandı ve kod adım adım çalıştırılarak hata ayıklama işlemleri gerçekleştirildi. Bu sayede kodun nasıl çalıştığı daha detaylı şekilde gözlemlendi.



4.Asenkron Programlama

Asenkron programlama, uzun süren işlemlerin programın akışını durdurmadan arka planda çalışmasını sağlayan bir yapıdır. Node.js tek thread çalışmasına rağmen bu yapı sayesinde aynı anda birden fazla işlemi yönetebilir.

Senkron:

- Kodlar sırayla çalışır
- Bir işlem bitmeden diğerine geçilmez
- Bekleme varsa program tamamen durur

Asenkron:

- Uzun süren işlemler arka planda çalışır
- Program diğer işlemlere devam eder
- Performans artışı sağlar

Örnek:

Senkron işlemler sırayla ve anında çalışırken, asenkron işlemler arka planda çalışır ve işlemleri tamamlandığında ekrana çıktı verir. Bu nedenle programın çıktısı yazım sırasına göre değil, işlemlerin tamamlanma süresine göre oluşur.

- console.log("start") → hemen çalışır
- console.log("stop") → hemen çalışır
- setTimeout(1) → 1saniyelik timeout (asenkron)
- setTimeout(2) → 2 saniyelik timeout (asenkron)

```
28  console.log("start");
29
30  setTimeout(() => {
31      console.log("2 minute timeout")
32  }, 2000);
33
34  setTimeout(()=>{
35      console.log("1 minute timeout");
36  },1000)
37  console.log("stop");
38
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\koray\Desktop\nodeproject> node .\lab.js
2 minute timer
1 minute timeout
PS C:\Users\koray\Desktop\nodeproject> node .\lab.js
start
stop
1 minute timeout
2 minute timeout
PS C:\Users\koray\Desktop\nodeproject> node .\lab.js
start
stop
1 minute timeout
2 minute timeout
```

5. Event Loop Mantığı

Asenkron yapının nasıl çalıştığını anlamak için event loop mantığı incelendi.

Temel bileşenler:

- Call Stack: Çalışan fonksiyonların bulunduğu yer
- Node API: Asenkron işlemlerin yürütüldüğü alan
- Callback Queue: İşlemi tamamlanan fonksiyonların sıraya alındığı yer

Çalışma mantığı:

1. Senkron işlemler doğrudan call stack üzerinde çalışır
2. Asenkron işlemler Node API tarafına gönderilir
3. İşlem tamamlanınca callback queue'ya alınır
4. Call stack boşaldığında, sıradaki işlem stack'e alınır ve çalıştırılır

Bu yapı sayesinde Node.js aynı anda birden fazla işlemi verimli şekilde yönetebilir.